Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Αναφορά 3ου εργαστηρίου
“Ψηφιακά Συστήματα VLSI”

ΟΝΟΜΑΤΕΠΩΝΥΜΑ	**Χαλβατζόγλου Ιωάννα** (AM: el21906) **Αϊφαντή Ελισάβετ** (AM: el22821)
ΟΜΑΔΑ	Ομάδα 13
ΔΙΔΑΣΚΟΝΤΕΣ	Κ.Πεκμεστζή Δ.Σούντρης Η. Παπαλάμπρου

Αθήνα
2026

MAC Unit:

- ***VHDL code***

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_unsigned.ALL;

-- Multiplier Accumulator unit
entity MAC is
   generic (
      coeff_width: integer := 8;
      data_width : integer := 8;
      maximum_vector_length : integer := 19 -- coeff_width + data_width + log2(tap number: 8)
   );
   Port (
      clk : in std_logic;
      rom_out : in std_logic_vector (coeff_width-1 downto 0);
      ram_out : in std_logic_vector (data_width-1 downto 0);
      mac_init, valid_out : in std_logic;
      y_out : out std_logic_vector (maximum_vector_length - 1 downto 0)
   );
end MAC;

architecture Behavioral of MAC is
   signal feedback : std_logic_vector (maximum_vector_length - 1 downto 0) := (others => '0');
begin

   -- 2 cycle LATENCY per accumulation
   -- 1 cycle read from rom and ram
   -- 1 cycle to multiply and add

   process(clk)
   begin
      if (clk'event and clk = '1') then
         if mac_init = '1' then
            feedback <= (others => '0');
            feedback(coeff_width + data_width - 1 downto 0) <= rom_out * ram_out; -- begin next
accumulation
         else
            feedback <= feedback + rom_out * ram_out;
         end if;

         if valid_out = '1' then
            y_out <= feedback;
         end if;
```

```
        end if;
    end process;

end Behavioral;
```

ROM:

- ***VHDL code***

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.std_logic_unsigned.all;

entity mlab_rom is
            generic (
                    coeff_width : integer :=8                                   --- width of coefficients
(bits)
            );
    Port ( clk : in  STD_LOGIC;
                                en : in  STD_LOGIC;                             --- operation enable
           addr : in  STD_LOGIC_VECTOR (2 downto 0);                            -- memory address
           rom_out : out  STD_LOGIC_VECTOR (coeff_width-1 downto 0));           -- output data
end mlab_rom;

architecture Behavioral of mlab_rom is

    type rom_type is array (7 downto 0) of std_logic_vector (coeff_width-1 downto 0);
    signal rom : rom_type:= ("00001000", "00000111", "00000110", "00000101", "00000100",
"00000011", "00000010",
                          "00000001");                                          -- initialization of rom with user
data

    signal rdata : std_logic_vector(coeff_width-1 downto 0) := (others => '0');
begin

    rdata <= rom(conv_integer(addr));

    process (clk)
    begin
        if (clk'event and clk = '1') then
            if (en = '1') then
                rom_out <= rdata;
            end if;
        end if;
    end process;
end Behavioral;
```

RAM:

- ***VHDL code***

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity ram is
          generic (
                  data_width : integer :=8                                  --- width of data (bits)
          );
   port (clk  : in std_logic;
        rst  : in std_logic;
        we   : in std_logic;                                                --- memory write enable
              en   : in std_logic;                                  --- operation enable
        addr : in std_logic_vector(2 downto 0);                     -- memory address
        di   : in std_logic_vector(data_width-1 downto 0);          -- input data
        do   : out std_logic_vector(data_width-1 downto 0));        -- output data
end ram;

architecture Behavioral of ram is

   type ram_type is array (7 downto 0) of std_logic_vector (data_width-1 downto 0);
   signal RAM : ram_type := (others => (others => '0'));       -- initialize at 0

begin
   process (clk, rst)
   begin
     if rst = '1' then
        RAM <= (others => (others => '0'));          -- reset to 0
      else
        if clk'event and clk = '1' then
          if en = '1' then
            if we = '1' then                                -- write operation
              RAM(7 downto 1) <= RAM(6 downto 0);
              RAM(conv_integer(addr)) <= di;
              do <= di;
             else                                                   -- read operation
               do <= RAM(conv_integer(addr));
             end if;
           end if;
        end if;
      end if;
   end process;

end Behavioral;
```

Control Unit:

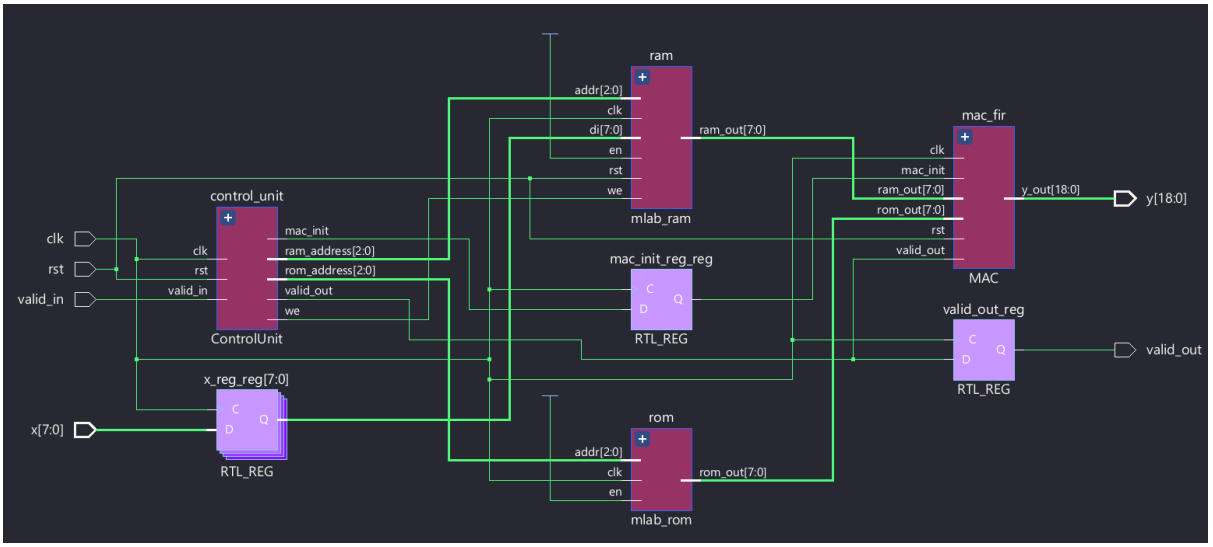
- ***VHDL code***

```
library IEEE;
use IEEE.std_logic_1164.ALL;
use IEEE.std_logic_unsigned.ALL;

entity control_unit is
   Port (
      clk         : in std_logic;
      rst         : in std_logic;
      valid_in    : in std_logic;
      rom_address  : out std_logic_vector(2 downto 0);
      ram_address  : out std_logic_vector(2 downto 0);
      mac_init    : out std_logic;
      we          : out std_logic;
      valid_out   : out std_logic
   );
end control_unit;

architecture Behavioral of control_unit is

   signal counter : std_logic_vector(2 downto 0) := (others => '0');
   signal temp_valid_out : std_logic_vector(8 downto 0) := (others => '0');

begin

   process(clk)
   begin
      if rising_edge(clk) then

         -- RESET
         if rst = '1' then
            counter <= (others => '0');
            temp_valid_out <= (others => '0');
            mac_init <= '1';
            we <= '0';

         else

            -- MAC init & write enable
            if counter = "000" then
               mac_init <= '1';
               we <= valid_in;
            else
               mac_init <= '0';
```

```
            we <= '0';
        end if;

        -- addresses
        rom_address <= counter;
        ram_address <= counter;

        -- counter
        if not (valid_in = '0' and counter = "000") then
            counter <= counter + 1;
        end if;

        -- delay 9 clk cycles
        temp_valid_out(0) <= valid_in;
        temp_valid_out(8 downto 1) <= temp_valid_out(7 downto 0);
        valid_out <= temp_valid_out(8);

    end if;
  end if;
end process;

end Behavioral;
```

FIR:

- ***RTL Schematic***



❖ ***VHDL code***

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.std_logic_unsigned.ALL;

entity FIR is
   generic (
      coeff_width: integer := 8;
      data_width : integer := 8;
      maximum_vector_length : integer := 19 -- coeff_width + data_width + log2(tap number: 8)
   );
   Port (
      clk : in std_logic;
      rst : in std_logic;
      valid_in : in std_logic;
      x : in std_logic_vector(data_width - 1 downto 0);
      y : out std_logic_vector(maximum_vector_length - 1 downto 0);
      valid_out : out std_logic
   );
end FIR;

architecture Behavioral of FIR is
   component MAC is
      generic (
         coeff_width: integer := 8;
         data_width : integer := 8;
         maximum_vector_length : integer := 19 -- coeff_width + data_width + log2(tap number: 8)
      );
      Port (
         clk : in std_logic;
         rom_out : in std_logic_vector (coeff_width-1 downto 0);
         ram_out : in std_logic_vector(data_width-1 downto 0);
         mac_init, valid_out : in std_logic;
         y_out : out std_logic_vector (maximum_vector_length - 1 downto 0)
      );
   end component;

   component mlab_rom is
       generic (
         coeff_width : integer :=8                                    --- width of coefficients (bits)
       );
      Port ( clk : in  STD_LOGIC;
             en : in  STD_LOGIC;                                  --- operation enable
             addr : in  STD_LOGIC_VECTOR (2 downto 0);                          -- memory address
             rom_out : out  STD_LOGIC_VECTOR (coeff_width-1 downto 0));       -- output data
   end component;
```

```
    component mlab_ram is
        generic (
            data_width : integer :=8                                --- width of data (bits)
         );
        port (clk  : in std_logic;
              rst  : in std_logic;
              we   : in std_logic;                                  --- memory write enable
              en   : in std_logic;                                  --- operation enable
              addr : in std_logic_vector(2 downto 0);               -- memory address
              di   : in std_logic_vector(data_width-1 downto 0);    -- input data
              ram_out   : out std_logic_vector(data_width-1 downto 0));    -- output data
    end component;

    component ControlUnit is
        Port (
            clk, rst, valid_in : in std_logic;
            we, mac_init, valid_out : out std_logic;
            rom_address, ram_address : out std_logic_vector(2 downto 0)
        );
    end component;

    signal rom_out : std_logic_vector(coeff_width - 1 downto 0);
    signal ram_out: std_logic_vector(data_width - 1 downto 0);
    signal we, mac_init, mac_init_reg: std_logic := '0';
    signal rom_address, ram_address : std_logic_vector(2 downto 0) := (others => '0');
    signal x_reg : std_logic_vector(data_width - 1 downto 0);
    signal valid_out_reg : std_logic;

begin

    mac_fir: MAC port map (
        clk => clk,
        rom_out => rom_out,
        ram_out => ram_out,
        mac_init => mac_init_reg,
        valid_out => valid_out_reg,
        y_out => y
    );

    rom: mlab_rom port map (
        clk => clk,
        en => '1',
        addr => rom_address,
        rom_out => rom_out
    );

    ram : mlab_ram port map (
```

```
        clk => clk,
        rst => rst,
        we => we, -- when we = '1' which is set by the control unit when input is valid, ram can start
writing to mac
        en => '1',
        addr => ram_address,
        di => x_reg,
        ram_out => ram_out
    );

    control_unit: ControlUnit port map (
        clk => clk,
        rst => rst,
        valid_in => valid_in,
        we => we,
        mac_init => mac_init,
        valid_out => valid_out_reg,
        rom_address => rom_address,
        ram_address => ram_address
    );

    process(clk)
    begin
        if rst = '1' then
            x_reg <= (others => '0');
            mac_init_reg <= '0';
        else
            if rising_edge(clk) then
            x_reg <= x;
            mac_init_reg <= mac_init;    -- delay mac_init by one cycle for rom and ram to send data to
mac
            end if;
        end if;
    end process;

end Behavioral;
```

❖ ***Testbench***

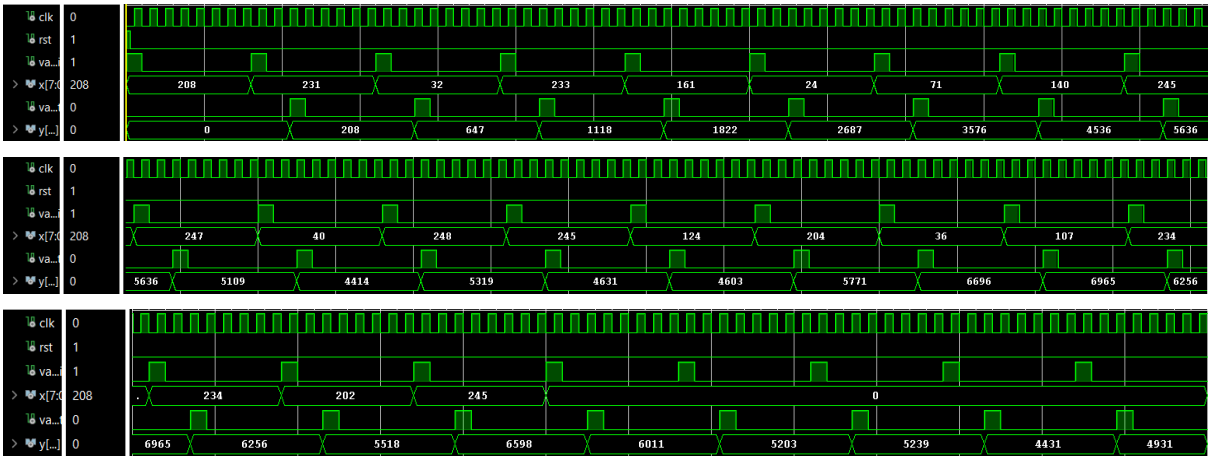
```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
use IEEE.NUMERIC_STD.ALL;
use IEEE.math_real.all;

entity fir_tb is
end fir_tb;

architecture testbench of fir_tb is
   constant data_width, coeff_width : integer := 8;
   constant maximum_vector_length : integer := 19;

   component fir is
      Port (
         clk : in std_logic;
         rst : in std_logic;
         valid_in : in std_logic;
         x : in std_logic_vector(data_width - 1 downto 0);
         y : out std_logic_vector(maximum_vector_length - 1 downto 0);
         valid_out : out std_logic
      );
   end component;

   --Input Signals
   signal clk: std_logic:='0';
   signal rst: std_logic:='0';
   signal valid_in: std_logic:='0';
   signal x : std_logic_vector(8-1 downto 0):=(others=>'0');
   signal valid_out: std_logic:='0';

   --Output Signals
   signal y : std_logic_vector(19-1 downto 0):=(others=>'0');

   --Clock
   constant CLK_PERIOD : time := 10ns;

   begin

      clk_process: process
      begin
         clk <= '0';
         wait for CLK_PERIOD/2;
         clk <= '1';
         wait for CLK_PERIOD/2;
```

```
end process;



UUT: fir port map (
   clk => clk,
   rst => rst,
   valid_in => valid_in,
   x => x ,
   y => y,
   valid_out => valid_out
);

TEST:
   process
   begin
      rst <= '1';
      valid_in <= '1';
      x <= std_logic_vector(to_unsigned(208, 8));
      wait for CLK_PERIOD/4;
      rst <= '0';
      wait for 3*CLK_PERIOD/4;
      valid_in <= '0';
      wait for 7*CLK_PERIOD;

      valid_in <= '1';
      x <= std_logic_vector(to_unsigned(231, 8));
      wait for CLK_PERIOD;
      valid_in <= '0';
      wait for 7*CLK_PERIOD;

      valid_in <= '1';
      x <= std_logic_vector(to_unsigned(32, 8));
      wait for CLK_PERIOD;
      valid_in <= '0';
      wait for 7*CLK_PERIOD;

      valid_in <= '1';
      x <= std_logic_vector(to_unsigned(233, 8));
      wait for CLK_PERIOD;
      valid_in <= '0';
      wait for 7*CLK_PERIOD;

      valid_in <= '1';
      x <= std_logic_vector(to_unsigned(161, 8));
      wait for CLK_PERIOD;
      valid_in <= '0';
      wait for 7*CLK_PERIOD;
```

```
valid_in <= '1';
x <= std_logic_vector(to_unsigned(24, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(71, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(140, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(245, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(247, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(40, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(248, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;

valid_in <= '1';
x <= std_logic_vector(to_unsigned(245, 8));
wait for CLK_PERIOD;
valid_in <= '0';
wait for 7*CLK_PERIOD;
```

```
        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(124, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;

        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(204, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;

        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(36, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;

        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(107, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;

        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(234, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;

        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(202, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;

        valid_in <= '1';
        x <= std_logic_vector(to_unsigned(245, 8));
        wait for CLK_PERIOD;
        valid_in <= '0';
        wait for 7*CLK_PERIOD;
    wait;

    end process;

end testbench;
```

❖ *Post-Implementation Functional Simulation*



Το FIR φίλτρο λειτουργεί στο στάδιο Post-Implementation κατάλληλα, χωρίς undefined ή high impedance σήματα. Η έξοδος y παράγεται ακριβώς κάθε 8 κύκλους ρολογιού ως η συνέλιξη των εισόδων x με τους συντελεστές h στη ROM, ενώ ενεργοποιείται το σήμα valid_out, στον ίδιο κύκλο ρολογιού που το y λαμβάνει το τελικό αποτέλεσμα για τη νέα είσοδο.

❖ *Resources και Critical Path*

Για την υλοποίηση του φίλτρου FIR στο FPGA, χρειάζονται συνολικά **88 Look Up Tables και 142 Flip Flops**. Καθώς το φίλτρο είναι μικρής κλίμακας, δεν χρειάστηκαν DSP slices ή BRAM/URAM blocks.

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP
✓ synth_1	constrs_1	synth_design Complete!												88	142	0	0	0
✓ impl_1	constrs_1	route_design Complete!	NA	NA	NA	NA		NA	18.080	0	142 CW			88	142	0	0	0

Το κρίσιμο μονοπάτι έχει καθυστέρηση **7.577nS**. Αποτελεί το μονοπάτι από το clock input pin του rom_out[2] στο data input pin του καταχωρητή feedback_reg[17].

Name	Slack	Levels	High Fanout	From	To	Total ...	Logic Delay	Net Delay	Requirement	Source Clock
Path 1	∞	10	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[17]/D	7.577	2.686	4.891	∞	
Path 2	∞	10	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[18]/D	7.321	2.597	4.724	∞	
Path 3	∞	10	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[16]/D	7.306	2.571	4.735	∞	
Path 4	∞	9	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[9]/D	7.175	2.835	4.340	∞	
Path 5	∞	9	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[8]/D	7.140	2.718	4.422	∞	
Path 6	∞	9	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[10]/D	6.886	2.718	4.168	∞	
Path 7	∞	9	20	rom/rom_out_reg[2]/C	mac_fir/feedback_reg[11]/D	6.631	2.800	3.831	∞	

Από το κρίσιμο μονοπάτι, η μέγιστη συχνότητα λειτουργίας του FPGA, υπολογίζεται ως:

$$f_{max} = \frac{1}{t_{delay}} = 131.98MHz$$

Ωστόσο, για την ευσταθή λειτουργία του κυκλώματος, θα επιλεγόταν μικρότερη συχνότητα από τη μέγιστη. Τα timing delays μπορεί να διαφέρουν λόγω των διακυμάνσεων PVT αλλά και για να περιοριστεί η κατανάλωση ισχύος, μένοντας εντός των προδιαγραφών λειτουργίας.